

pythia: a Lean 4 tactic library for statistical proof automation

Anonymous Authors¹

Abstract

Lean 4 with Mathlib ships strong general automation (`aesop`, `simp`, `linarith`, `polyrith`, `nlinarith`, `positivity`, `measurability`, `bound`, `gcongr`). It does not ship a statistics-domain hammer. We present *pythia*, a Lean 4 tactic library that closes statistical-domain goals via five registered tactics: *pythia* (general dispatcher), `stats_ineq` (concentration and tail-bound inequalities), `prob_simp` (probability normalization rewrites), `anytime_valid` (Ville-bound closer for confidence sequences), and `z3_check` (Z3 oracle with `linarith` reconstruction in the CoqHammer style of Czapka & Kaliszyk (2018)). The library ships with extensible attribute-driven registries (`@[stat_lemma]`, `@[stats_ineq]`, `@[prob_simp]`) and a named `aesop` ruleset *Pythia*. A library of approximately twelve modules covers anytime-valid confidence sequences (Howard et al., 2021; Waudby-Smith & Ramdas, 2024), matrix concentration scaffolds (Tropp, 2012; Lieb, 1973), e-detectors (Shin et al., 2024; Ramdas et al., 2023), queueing primitives, control Lyapunov bounds, and the binary Bretagnolle-Huber inequality. Every shipped theorem is axiom-clean against the standard Lean kernel set (`propext`, `Classical.choice`, `Quot.sound`). Hard structural Mathlib gaps are routed to the Aristotle automated theorem prover (Harmonic, 2025) as an oracle, with kernel-checked reconstruction inside Lean and a refusal policy for vacuous-truth closures. The library is pinned to Lean 4.28 + Mathlib v4.28.0 and released open-source. Measured on a small subset of the FormalAVS benchmark (Anonymous, 2026), Aristotle as oracle closes

problems unreachable by hand-Mathlib alone. Full numbers are reported in the dataset paper.

1. Introduction

Lean 4 with Mathlib ships strong general-purpose proof automation: `aesop` for forward search, `simp` for rewriting, `linarith` and `polyrith` for linear and polynomial arithmetic, `nlinarith` for non-linear arithmetic, `positivity` for sign goals, `measurability` for the σ -algebra fragment, `bound` for monotone composition, `gcongr` for generalized congruence (de Moura & Ullrich, 2021; The Mathlib Community, 2020; 2025). Statistical proofs route through these general tactics. Each statistical lemma typically requires by-hand decomposition into a chain of `simp` sets, `linarith` hypotheses, and Mathlib lemma selection. There is no domain hammer for statistics.

The closest design template is CoqHammer (Czapka & Kaliszyk, 2018): an external ATP (Vampire, E, CVC4) discharges goals with kernel-checked reconstruction inside Coq. Sledgehammer (Paulson & Blanchette, 2010; Blanchette et al., 2011) plays the same role in Isabelle/Higher-Order Logic (HOL). Lean 4 lacks a comparable Sledgehammer-class tool for general proof, and lacks a statistics-domain hammer specifically. We do not address the general case in this paper. We address the statistics-domain case.

Contribution. We present *pythia*, a Lean 4 tactic library for statistical proof automation. *pythia* ships five registered tactics, one named `aesop` ruleset, three attribute-driven registries, and a twelve-module proof library covering anytime-valid confidence sequences, matrix and scalar concentration, e-detectors, queueing, control, and information-theoretic divergences. The shipped library is axiom-clean against `{propext, Classical.choice, Quot.sound}` and pinned to Lean 4.28 + Mathlib v4.28.0. Hard structural goals are routed to Aristotle (Harmonic, 2025) as an oracle, with kernel-checked reconstruction inside Lean.

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

Case study (vacuous-truth catch). On 2026-04-26 the Aristotle prover closed a candidate theorem in our `wald_wolfowitz_optimal` development by routing through a contradictory premise: a `Bool` partition combined with a log-boundary constraint forced both $\alpha \geq 1$ and $\alpha < 1$, closing the goal vacuously. The closure was kernel-checked. The theorem statement was the bug. `pythia`’s discipline (axiom-clean only, refuse vacuous closures, audit hypotheses) caught the issue and forced a restatement to the two-measure form. We treat this kind of catch as a first-class library responsibility, not as a separate review pass.

Paper organisation. Section 2 describes the five-tactic surface and the registry attributes. Section 3 groups the shipped library content by domain. Section 4 describes the Aristotle integration as an oracle and the vacuous-truth refusal policy. Section 5 compares against `aesop`, `Sledgehammer`, `CoqHammer`, and `Mathlib`’s existing tactic suite, states limitations, and reports a small Aristotle vs DSPv2 vs Opus 4.6 head-to-head on a subset of FormalAVS (Anonymous, 2026). Full numbers are in the dataset paper.

Reproduction and code. The `pythia` library is open source. The library link, axiom audit transcripts, and reproduction recipes are listed in Appendix D. The dataset of statistical theorem-proving benchmarks (FormalAVS, Anonymous (2026)) is reported in a separate paper.

2. The pythia tactic surface

`pythia` ships five user-facing tactics, one named `aesop` ruleset, and three attribute-driven registries. The design follows the `CoqHammer` pattern (Czajka & Kaliszyk, 2018): a domain-specific dispatcher with extensible attribute-driven lemma selection, a fall-through to general-purpose automation, and kernel-checked reconstruction for any external-oracle steps.

2.1. Five tactics

`pythia` (top-level dispatcher). The default entry point. Tries the registered statistical lemma database, then the named `aesop` ruleset `Pythia`, then `linarith`. The fall-through ordering is fixed. The lemma database and `aesop` ruleset are user-extensible.

`stats_ineq`. Discharges sub-Gaussian, sub-gamma, Bernstein-Bennett, and Hoeffding-shape inequalities. The lemma set is registered via `@[stats_ineq]`. Reconstruction is internal Lean and axiom-clean.

`prob_simp`. Probability-domain rewrite set. Pushes integrals through linear operators, splits indicator products, and reduces martingale expressions to MGF form for down-

Tactic	Role
<code>pythia</code>	Top-level dispatcher. Tries registered <code>stat_lemmas</code> , then <code>aesop</code> ruleset <code>Pythia</code> , then <code>linarith</code> .
<code>stats_ineq</code>	Discharges concentration and tail-bound inequalities (Hoeffding, Bernstein, sub-Gaussian, sub-gamma).
<code>prob_simp</code>	Probability normalization rewrites. Pushes integrals, splits indicator products, reduces to MGF form.
<code>anytime_valid</code>	Closes Ville-bound goals for anytime-valid confidence sequences. Specializes to Howard-Ramdas, betting, vector.
<code>z3_check</code>	Linear-real-arithmetic Z3 oracle with <code>linarith</code> reconstruction. Kernel-checked.

Table 1. The five `pythia` tactics. `pythia` is the default entry point. The four specialized tactics are exposed for direct invocation when the user knows the goal shape.

stream use by `stats_ineq`. The rewrite set is registered via `@[prob_simp]`.

`anytime_valid`. Closes Ville-type maximal-inequality goals. Specializes to the Howard-Ramdas (Howard et al., 2021), betting (Waudby-Smith & Ramdas, 2024), and vector (Whitehouse et al., 2025) families, dispatching by goal shape. The case-split table is small (currently four families) and hard-coded. Future families register via `@[anytime_valid]`.

`z3_check`. External Z3 oracle for linear real arithmetic, with `linarith` reconstruction inside Lean. Z3 supplies the proof witness. `linarith` reconstructs the witness step inside the kernel. The kernel is the final word. Z3 cannot smuggle axioms. This follows the `CoqHammer` reconstruction pattern (Czajka & Kaliszyk, 2018).

2.2. Registry attributes

The library uses three attribute-driven registries:

- `@[stat_lemma]`: lemma database for the top-level `pythia` dispatcher.
- `@[stats_ineq]`: tail-bound and concentration lemmas dispatched by `stats_ineq`.
- `@[prob_simp]`: probability rewrite rules dispatched by `prob_simp`.

User extensions consist of tagging library lemmas with the appropriate attribute. No fork is required.

2.3. The Pythia aesop ruleset

The library declares a named aesop ruleset `Pythia` containing the registered `stat_lemma` entries. Direct invocation is supported via `aesop (rule_sets := [Pythia])`, equivalent to the `pythia` dispatcher’s middle stage. This gives users a finer-grained interface when the dispatcher’s outer wrapping is not desired.

2.4. Architecture

Architecture (TODO: replace with TikZ). The `pythia` dispatcher tries the `@[stat_lemma]` database first, then the `Pythia` aesop ruleset, then `linarith`. Each stage is independently Lean-kernel-checked. `z3_check` is a separate user-invoked tactic with its own `linarith`-based reconstruction step, not part of the dispatcher chain.

Figure 1. The `pythia` dispatcher chain. Three stages, each kernel-checked independently. `z3_check` sits outside the chain.

Why this shape. The three-stage chain mirrors the empirical structure of statistical proofs in our library: many goals close on a single lemma application (stage 1), some require forward search through a lemma set (stage 2), some reduce to a linear arithmetic obligation after the rewrite (stage 3). Routing by goal shape is not necessary at the top level. The cost of trying stages in order is small relative to the cost of incorrect early termination.

3. Library content

The shipped `pythia` library covers approximately twelve modules grouped by domain. Every theorem in the modules listed here is axiom-clean against `{propext, Classical.choice, Quot.sound}` under the Lean 4.28 + Mathlib v4.28.0 toolchain pin. We summarise groups. The appendix lists every theorem.

3.1. Anytime-valid inference

Howard-Ramdas confidence sequences (Howard et al., 2021), betting-CS (Waudby-Smith & Ramdas, 2024), and the vector-valued extension (Whitehouse et al., 2025) are each backed by a Ville-bound specialization in `anytime_valid`. The `HowardRamdasCS` and `BettingCS` modules expose the admissibility certificates of Ramdas et al. (2023).

3.2. Concentration and tail bounds

`stats_ineq` dispatches sub-Gaussian and sub-gamma tail bounds, the Bernstein-Bennett MGF bridge, and Hoeffding’s inequality for bounded random variables. The MGF

bridge composes with `prob_simp` for the integral side and with general Mathlib infrastructure (`measurability`, `integrability`) for side conditions.

3.3. E-detectors and merging

The e-process formalism of Ramdas et al. (2023); Shin et al. (2024) yields uniform-time validity by combining e-processes via averaging. The `combine_eprocesses_avg` lemma is the workhorse. The surrounding scaffold registers average-merge and product-merge variants. Closure of the average-merge case is axiom-clean.

3.4. Matrix concentration (scaffold)

Matrix Bernstein (Tropp, 2012) and the Lieb concavity inequality (Lieb, 1973) are present as scaffold modules. The matrix-Bernstein head theorem currently lands as a typed statement with key obligations marked. The dependent σ -additivity step routes through Aristotle as oracle (see Section 4).

3.5. Other domains

- Control: `LyapunovDiscrete`, discrete-time Lyapunov stability for linear systems.
- Queueing: `ErlangB`, `LittleLaw`.
- Time series: `Wold` (Wold, 1938).
- Risk: conditional value-at-risk (Rockafellar & Uryasev, 2000).
- Asymptotics: scalar and multivariate delta method.
- Information theory: Bretagnolle-Huber inequality (binary case).

3.6. Five highlights

We name five representative results. Detailed statements appear in Appendix A.

1. **Howard-Ramdas Ville bound.** The maximal inequality for the self-normalized confidence sequence (Howard et al., 2021). Closes via `anytime_valid`.
2. **Betting-CS coverage.** The supermartingale coverage certificate of Waudby-Smith & Ramdas (2024). Axiom-clean closure.
3. **Bernstein-Bennett MGF bridge.** Routes between the variance-aware sub-gamma tail and the Bennett MGF form.
4. **combine_eprocesses_avg.** Average-merge of e-processes (Ramdas et al., 2023; Shin et al., 2024). Axiom-clean closure with full reconstruction.

5. Bretagnolle-Huber binary. The binary-case total-variation lower bound on testing risk. Standard in information-theoretic lower-bound arguments.

Pinning. The library is pinned to Lean 4.28 + Mathlib v4.28.0. The pin is necessary: the modules consume Mathlib’s measure theory and probability subdirectories, which evolve quickly. We track Mathlib release tags rather than master for reproducibility.

4. Aristotle integration as oracle

A subset of statistical proof goals exceed what `aesop`, `simp`, `linarith`, and the registered `pythia` lemma sets close in practice. We route these to the Aristotle automated theorem prover (Harmonic, 2025) as an oracle. Aristotle returns a Lean 4 proof script. The script is checked by the Lean kernel inside the calling project. Aristotle is one of two oracles in the library. The other is Z3 (Section 2.1).

4.1. Submission pattern

Each Aristotle call is a project tarball submission. The flow: state the theorem in Lean with a `sorry`, package the project without the Lean build cache (`.lake` excluded), submit via `aristotle submit`, monitor for completion, integrate the returned proof, and run the axiom audit. The axiom audit is the verification step that distinguishes a kernel-checked Aristotle proof from a vacuous closure.

4.2. Vacuous-truth refusal

ATPs can close a theorem by deriving a contradiction from its hypotheses, then concluding anything. The kernel accepts such proofs. The proof is sound. The theorem statement is the bug.

Case (2026-04-26). We submitted a candidate `wald.wolfowitz.optimal` theorem. Aristotle returned a closure that combined a `Bool` partition with a log-boundary constraint to force both $\alpha \geq 1$ and $\alpha < 1$. The kernel accepted the proof. The audit flagged the hypothesis chain as self-contradictory. We rejected the closure, restated the theorem in two-measure form (separating the null and alternative measures explicitly), and resubmitted. The two-measure form was the intended target. The original `Bool`-partition form was a statement bug.

Audit policy. The library treats vacuous closures as theorem-statement bugs, not as prover successes. Concretely: every Aristotle-returned proof is audited for contradictory premises before integration. Closures flagged as vacuous block the corresponding theorem from the axiom-clean library until the statement is reformed. This policy is not unique to Aristotle. The same audit applies to any

external prover, including human proofs that close via a hypothesis contradiction the author did not notice.

4.3. Z3 as a second oracle

The `z3.check` tactic dispatches linear-real-arithmetic goals to Z3 (de Moura & Bjørner, 2008). Z3 returns a proof witness. The witness is then reconstructed inside Lean by `linarith`. The Lean kernel is the final word. The pattern is the CoqHammer (Czajka & Kaliszyk, 2018) reconstruction strategy adapted to Lean 4. Z3 cannot smuggle axioms. `linarith` is axiom-clean and the reconstruction step certifies the witness. If `linarith` cannot reconstruct, the `z3.check` call fails loud rather than silently trusting Z3.

4.4. When to call Aristotle

The library convention is to attempt local closure first. If the goal closes by `rfl`, `simp`, `positivity`, `linarith`, `nlinarith`, `polyrith`, or the registered `stat.lemma` database, no Aristotle call is made. Aristotle is reserved for hard structural Mathlib gaps: measure-theoretic σ -additivity steps, dominated-convergence side conditions, matrix-concentration intermediate lemmas, and similar obligations for which a hand-Mathlib closure either does not exist or would exceed five lines of dispatch. The five-line threshold is a convention, not a hard cap. The vacuous-closure audit is the gate on integration. The dispatch threshold is only the call decision.

5. Comparison and discussion

5.1. Comparison to existing automation

aesop. Aesop (Limperg & From, 2023) is the workhorse forward-search tactic in Lean 4. `pythia` declares a named aesop ruleset and uses aesop in stage 2 of the dispatcher chain. `pythia` is not a replacement for aesop. It is a statistics-domain registry plus a Z3 oracle plus a vacuous-closure audit, layered on top of aesop.

Sledgehammer. Sledgehammer (Paulson & Blanchette, 2010; Blanchette et al., 2011) dispatches Isabelle/HOL goals to external ATPs (Vampire, E, Z3, CVC4) and reconstructs the proof. There is no direct Lean 4 equivalent. `pythia` addresses the statistics-domain case via `z3.check` for linear-real-arithmetic and Aristotle as a structural oracle. A general-purpose Lean Sledgehammer is out of scope for this paper.

CoqHammer. CoqHammer (Czajka & Kaliszyk, 2018) dispatches Coq goals to external ATPs with kernel-checked reconstruction. `z3.check` in `pythia` adopts the same reconstruction template (witness from external prover, in-kernel checking via `linarith`). CoqHammer is the design

template. We do not claim a Lean-side reimplementa-
 CoqHammer’s full premise selection.

Mathlib’s existing tactic suite. Mathlib supplies
 linarith, polyrith, nlinarith, positivity,
 measurability, bound, gcgr. None of these is
 a statistics-domain hammer. pythia composes with all
 of them. The fall-through to linarith in stage 3 of the
 dispatcher chain is the explicit composition.

5.2. Aristotle vs DSPv2 vs Opus 4.6 (illustrative)

Table 2 reports a small subset of FormalAVS problems
 (Anonymous, 2026) on which Aristotle, DSPv2 (a fine-
 tuned local-GPU drafter), and Opus 4.6 (an API model)
 were each given a single attempt. The full benchmark num-
 bers are in the dataset paper. The point of this subset is
 to illustrate the regime in which Aristotle as oracle closes
 problems that hand-Mathlib alone does not. Per-problem
 solve outcomes (close or open) are kernel-checked.

Problem	Aristotle	DSPv2	Opus 4.6
combine_eprocesses_avg	TODO	TODO	TODO
ville_bound_betting	TODO	TODO	TODO
matrix_bernstein_step	TODO	TODO	TODO
bretagnolle_huber_binary	TODO	TODO	TODO

Table 2. Illustrative subset of FormalAVS. Solve outcomes (close
 vs open) per single attempt. Full benchmark numbers in Anony-
 mous (2026).

The wider table on the full benchmark subset is in Ap-
 pendix C.

5.3. Limitations

Three concrete gaps. (i) Domain scope: pythia is
 statistics-domain only. It does not address general proof
 automation in Lean 4. (ii) Toolchain pin: the library ships
 pinned to Lean 4.28 + Mathlib v4.28.0. Mathlib evolves
 quickly. Each major Mathlib release typically requires a
 porting pass. (iii) Aristotle dependency: the hardest clo-
 sures route to a proprietary external prover. The z3.check
 oracle is open-source. The Aristotle oracle is not. The li-
 brary is usable without Aristotle for any goal that closes via
 the pythia dispatcher chain or z3.check. A subset of
 structural Mathlib gaps requires either Aristotle or a hand
 proof.

References

Anonymous. FormalAVS: a benchmark for statistical theo-
 rem proving in Lean 4, 2026. Companion dataset paper,
 anonymized for double-blind review.

Blanchette, J. C., Böhme, S., and Paulson, L. C. Extending

Sledgehammer with SMT solvers. In *23rd International
 Conference on Automated Deduction (CADE)*, pp. 116–
 130, 2011. doi: 10.1007/978-3-642-22438-6_11.

Czajka, Ł. and Kaliszyk, C. Hammer for Coq: Au-
 tomation for dependent type theory. *Journal of Auto-
 mated Reasoning*, 61(1-4):423–453, 2018. doi: 10.1007/
 s10817-018-9458-4.

de Moura, L. and Bjørner, N. Z3: An efficient SMT solver.
 In *14th International Conference on Tools and Algorithms
 for the Construction and Analysis of Systems (TACAS)*, pp.
 337–340, 2008. doi: 10.1007/978-3-540-78800-3_24.

de Moura, L. and Ullrich, S. The Lean 4 theorem prover and
 programming language. In *28th International Conference
 on Automated Deduction (CADE)*, pp. 625–635, 2021.
 doi: 10.1007/978-3-030-79876-5_37.

Harmonic. Aristotle: an automated theorem prover for
 Lean. <https://harmonic.fun/>, 2025. Proprietary
 commercial tool.

Howard, S. R., Ramdas, A., McAuliffe, J., and Sekhon, J.
 Time-uniform, nonparametric, nonasymptotic confidence
 sequences. *Annals of Statistics*, 49(2):1055–1080, 2021.
 doi: 10.1214/20-AOS1991.

Lieb, E. H. Convex trace functions and the Wigner-Yanase-
 Dyson conjecture. *Advances in Mathematics*, 11(3):267–
 288, 1973. doi: 10.1016/0001-8708(73)90011-X.

Limperg, J. and From, A. H. Aesop: White-box best-
 first proof search for Lean. In *Proceedings of the 12th
 ACM SIGPLAN International Conference on Certified
 Programs and Proofs (CPP)*, pp. 253–266, 2023. doi:
 10.1145/3573105.3575671.

Paulson, L. C. and Blanchette, J. C. Three years of experi-
 ence with Sledgehammer, a practical link between auto-
 matic and interactive theorem provers. In *Proceedings of
 the 8th International Workshop on the Implementation of
 Logics (IWIL)*, 2010.

Ramdas, A., Grünwald, P., Vovk, V., and Shafer, G. Game-
 theoretic statistics and safe anytime-valid inference. *Sta-
 tistical Science*, 38(4):576–601, 2023.

Rockafellar, R. T. and Uryasev, S. Optimization of condi-
 tional value-at-risk. *Journal of Risk*, 2(3):21–41, 2000.

Shin, J., Ramdas, A., and Rinaldo, A. E-detectors: a non-
 parametric framework for sequential change detection.
New England Journal of Statistics in Data Science, 2024.

The Mathlib Community. The Lean mathematical library.
 In *Proceedings of the 9th ACM SIGPLAN International
 Conference on Certified Programs and Proofs (CPP)*, pp.
 367–381, 2020. doi: 10.1145/3372885.3373824.

The Mathlib Community. Mathlib: the Lean 4 mathematical library. <https://github.com/leanprover-community/mathlib4>, 2025. Repository tagged at v4.28.0 for the pythia pin.

Tropp, J. A. User-friendly tail bounds for sums of random matrices. *Foundations of Computational Mathematics*, 12(4):389–434, 2012. doi: 10.1007/s10208-011-9099-z.

Waudby-Smith, I. and Ramdas, A. Estimating means of bounded random variables by betting. *Journal of the Royal Statistical Society Series B*, 86(1):1–27, 2024.

Whitehouse, J., Ramdas, A., Wu, Z. S., and Sutton, R. Time-uniform self-normalized concentration for vector-valued processes. 2025.

Wold, H. *A Study in the Analysis of Stationary Time Series*. Almqvist & Wiksell, Stockholm, 1938.

A. Library inventory

The shipped library at the AI4MATH 2026 submission cut spans twelve modules. Theorems below are listed with module path, one-line statement, and axiom-audit status.

A.1. Anytime-valid inference

- `Pythia.AnytimeValid.HowardRamdasCS`: Howard-Ramdas Ville bound for self-normalized confidence sequences. (TODO: exact theorem name + statement.) Axiom-clean.
- `Pythia.AnytimeValid.BettingCS`: betting-CS coverage certificate. (TODO: exact.) Axiom-clean.
- `Pythia.AnytimeValid.combine_eprocesses_avg`: e-process average-merge. (TODO: exact.) Axiom-clean.

A.2. Concentration

- `Pythia.Concentration.SubGaussian`: sub-Gaussian tail bound (Hoeffding-style). Axiom-clean.
- `Pythia.Concentration.SubGamma`: sub-gamma tail bound. Axiom-clean.
- `Pythia.Concentration.BernsteinBennett`: variance-aware MGF bridge. Axiom-clean.

A.3. Matrix concentration (scaffold)

- `Pythia.Matrix.Lieb`: Lieb concavity inequality scaffold (Lieb, 1973). (TODO: which dependent obligations remain.)
- `Pythia.Matrix.Bernstein`: matrix-Bernstein scaffold (Tropp, 2012). (TODO.)

A.4. Other modules

- `Pythia.Control.LyapunovDiscrete`: discrete-time Lyapunov stability.
- `Pythia.Queueing.ErlangB`, `Pythia.Queueing.LittlesLaw`: queueing primitives.
- `Pythia.TimeSeries.Wold` (Wold, 1938).
- `Pythia.Risk.CVaR` (Rockafellar & Uryasev, 2000).
- `Pythia.Asymptotics.DeltaMethod`: scalar and multivariate.
- `Pythia.InfoTheory.BretagnolleHuber`: binary case.

B. Worked examples

B.1. anytime_valid: Ville-bound closer

B.2. z3_check: linear-real-arithmetic with reconstruction

B.3. pythia: chain-rule fall-through to aesop

B.4. combine_eprocesses_avg: axiom-clean closure walkthrough

C. Aristotle vs DSPv2 vs Opus 4.6 (wider subset)

Table 3 reports the wider subset.

Problem	Aristotle	DSPv2	Opus 4.6
TODO: populate from (Anonymous, 2026)			

Table 3. Wider subset from FormalAVS. Solve outcomes per single attempt. Full benchmark numbers in Anonymous (2026).

D. Reproduction

Toolchain pin. Lean 4.28 + Mathlib v4.28.0. The repository’s `lean-toolchain` file declares the exact toolchain. The Mathlib dependency in `lakefile.toml` pins the v4.28.0 release tag.

Build.

```
git clone <pythia-url-redacted>
cd pythia
lake exe cache get
lake build
```

Axiom audit. Each shipped module ends with a `#print axioms` block. The expected audit set is `{propext, Classical.choice, Quot.sound}`. Any deviation fails the audit.

FormalAVS benchmark. The full benchmark is reported in Anonymous (2026). Reproduction commands for the benchmark are in that paper’s reproduction appendix.

E. Related related-work, extended